

Reviewer Pack — Minimal Rank of Configuration Spaces Bounds Monomial Ideal C...

Entry 239251aecfea · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-28 21:58:58 UTC

Minimal Rank of Configuration Spaces Bounds Monomial Ideal Complexity

Entry ID: 239251aecfea **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-28 21:58:58 UTC

1. Conjecture

Field A (mathematical branch): Algebraic Topology (Configuration Spaces) **Field B** (complexity object): Complexity Theory (Monomial Ideal Complexity)

Statement:

[‘For every 3-CNF formula F with n variables, the minimal rank of its associated configuration space is lower-bounded by $2^{(n/3)}$.’, ‘This bound holds for all instances where $n \leq 40$.’, ‘The conjecture implies that the complexity of computing the complexity of monomial ideals increases exponentially with the size of the input.’]

Rationale (proposer’s reasoning):

[‘Configuration spaces have been studied in algebraic topology, but their application to complexity theory is novel. The connection between configuration spaces and monomial ideal complexity may reveal new insights into the computational hardness of problems.’, ‘If true, this conjecture would provide a direct link between topological properties and computational complexity, potentially leading to new algorithms or impossibility results.’, ‘The exponential lower bound suggests a deep structural relationship between the topology of configuration spaces and the complexity of monomial ideal computation.’]

Taxonomy category: TSEITIN_RES (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 63001b821219bef9

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, across 30 different random seeds, at least 80% of the computed ranks (logarithm base 2) fall within $\pm 1/3$ of $n/3$ for all $n \leq 40$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "configuration space" AND "minimal rank" AND "monomial ideal complexity" - "3-CNF formula" AND "configuration space" AND "complexity theory" - "exponential growth" AND "input size" AND "configuration spaces" AND "monomial ideals"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_3cnf(n):
        clauses = []
        for _ in range(n):
            literals = [random.choice([1, -1]) * (i + 1) for i in range(n)]
            clause = random.sample(literals, 3)
            clauses.append(clause)
        return clauses

    def configuration_space(clauses):
        n = len(clauses[0])
        space = []
        for assignment in itertools.product([0, 1], repeat=n):
            rank = 0
            for clause in clauses:
                if any(assignment[var - 1] == literal for literal in clause):
                    rank += 1
            space.append(rank)
        return space

    def min_rank(space):
        unique_ranks = set(space)
        return len(unique_ranks)
```

```

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    clauses = generate_3cnf(n)
    space = configuration_space(clauses)
    rank = min_rank(space)
    log2_rank = math.log2(rank) if rank > 0 else -math.inf
    expected_bound = n / 3
    within_bound = abs(log2_rank - expected_bound) <= 1/3 * expected_bound

    results.append({
        "n": n,
        "rank": rank,
        "log2_rank": log2_rank,
        "expected_bound": expected_bound,
        "within_bound": within_bound
    })

metric_value = sum(result["log2_rank"] for result in results) / len(results)
instances_tested = len(results)
conjecture_holds = all(result["within_bound"] for result in results)
counterexample = "" if conjecture_holds else "n/3 bound not met"

return {
    "metric_name": "log2(rank)",
    "metric_value": metric_value,
    "instances_tested": instances_tested,
    "conjecture_holds": conjecture_holds,
    "counterexample": counterexample
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30)) + list(map(lambda x: x*10, range(1, 10)))

    results = []
    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    mean_log_rank = sum(result["metric_value"] for result in results) / len(results)
    std_log_rank = math.sqrt(sum((result["metric_value"] - mean_log_rank) ** 2 for result in results) / len(results))
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_log_rank} std={std_log_rank} support_fraction={support_fraction}")
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next((seed for seed, result in zip(seeds, results) if not result["conjecture_holds"]))
        print(f"RESULT: FALSIFIED counterexample=\"n/3 bound not met\" first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=unknown")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c5b4f604.py", line 82, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c5b4f604.py", line 49, in run_trial
    space = configuration_space(clauses)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c5b4f604.py", line 35, in configuration
    if any(assignment[var - 1] == literal for literal in clause):
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_c5b4f604.py", line 35, in <genexpr>
    if any(assignment[var - 1] == literal for literal in clause):
           ^^^
```

NameError: name 'var' is not defined. Did you mean: 'vars'?

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed due to a NameError, which prevented it from producing any data. As the critic challenged the result and the pre-registered support condition was not unambiguously met, the verdict is INCONCLUSIVE. | next: Review the test code for the missing 'var' variable definition and fix the error before re-running the test.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10104
2	propose	ollama_remote	glm4:latest	0	0	10191
3	preregistration	ollama_remote	glm4:latest	0	0	5460
4	novelty	ollama_remote	glm4:latest	0	0	4780
5	novelty	ollama_remote	glm4:latest	0	0	5695
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	30095
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10131
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10262
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	9663
10	judge	ollama_remote	glm4:latest	0	0	10414

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 106796 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in research/audit/239251aecfea.jsonl)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in research/audit/239251aecfea.jsonl for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/239251aecfea.tar.gz (if generated)